

Tenant-Aware Enforcement in Multi-Tenant SaaS: JWT, Middleware, RLS, and Quotas

Pavan Sai Bodduluru | Student ID: X24211371

MSc Cloud Computing — National College of Ireland — Short Explanation Guide

Baseline: Cheng et al. 2025 VLDB Fair Transaction Processing — DOI 10.14778/3742728.3742751

Artefact: pavan/tenant-aware-enforcement/ | STATUS: FLOOR COMPLETE (LOCAL_SIM)

1. What is the project?

This MSc project builds a unified tenant-aware enforcement stack for multi-tenant SaaS. Four layers carry the same tenant identity end-to-end: JWT access tokens with `tenant_id` and role claims; application middleware that refuses cross-tenant resource access and filters ORM-style queries; PostgreSQL row-level security (RLS) on a shared schema; and Redis-style multi-dimensional quotas for API, storage, and compute. The floor artefact is `pavan/tenant-aware-enforcement/`. It runs locally with in-memory stand-ins, optional docker-compose for Postgres 16 and Redis 7, green pytest, and committed LOCAL_SIM JSON. Live AWS RDS/ElastiCache/API Gateway is not claimed yet.

One-sentence pitch: I stop cross-tenant data leaks and noisy neighbours by propagating tenant context from the token through middleware, the database, and atomic quotas — and I measure zero unauthorized access, Jain fairness, and latency budgets.

2. Problem statement + real-life example

Shared SaaS infrastructure creates two governance failures. Cross-tenant leakage means tenant A reads tenant B's rows when the application forgets a WHERE clause or an attacker forges a tenant claim. The noisy-neighbour effect means one heavy tenant starves others when quotas are missing or only cover HTTP rate limits. Identity, isolation, and quotas are often separate products, so tenant context is looked up repeatedly and inconsistently.

Apartment-building analogy: the keycard encodes which flat you belong to (JWT claims). The corridor guard checks the keycard against the door you try (middleware). Each flat's lock only opens for that flat's key even if a guard is bribed (RLS). The building's utility meters cut power when a flat exceeds its plan (Redis quotas). Cheng et al. 2025 improve how the building's lift scheduler shares capacity fairly; my work is the keycard-to-lock-to-meter chain.

- Security target: zero unauthorized cross-tenant access in 10,000 synthetic probes.
- Fairness target: Jain's index > 0.9 across tenants' consumed API quota.
- Latency targets: auth p95 < 50 ms; quota check p95 < 5 ms (CA1 budgets).

3. Why this matters in industry and governance

GDPR Articles 25 and 32 and ISO/IEC 27001 access-control controls treat isolation as a designed property, not a best-effort filter. Zero-trust guidance (NIST SP 800-207) wants

every request verified. Operators also need fair multi-dimensional limits so bursty but legitimate tenants are not treated identically to abusers without soft limits and visibility. This project makes those obligations testable in one artefact.

Honesty limits: LOCAL_SIM JWT HS256 and dict counters are not Cognito or ElastiCache; in-memory microseconds are not AWS p95; docker Postgres RLS is optional and not yet the measurement campaign. STATUS.md records LIVE_AWS=not_run. Responsible disclosure spirit (ISO/IEC 29147) applies: publish mitigations and outcomes, not weaponisable exploit recipes against third parties.

4. Baseline paper — problem, solution, gap, metrics

Primary baseline: Cheng, Kabcenell, Shi, Huey, Bailis, Crooks and Stoica (2025), Fair Transaction Processing for Multi-Tenant Databases, PVLDB 18(8), pp. 2602–2615, DOI 10.14778/3742728.3742751. OA PDF stored at pavan/baseline_papers/Cheng_et_al_2025_Fair_Txn_VLDB_baseline.pdf.

Problem: multi-tenant database tenants can starve each other under unfair transaction scheduling. Solution: DRFT-style fair scheduling with share guarantees and strategy-proofness inside the database engine. Metrics centre on fairness and throughput under contention.

Gap this CA closes: Cheng et al. operate at the transaction scheduler. They do not issue JWT tenant claims, enforce middleware filters, attach PostgreSQL RLS policies, or implement Redis multi-dimensional quotas with authentication and quota latency budgets plus a 10k cross-tenant probe suite. Wang et al. (2025) zero-trust dynamic access control (DOI 10.1186/s42400-024-00320-x) is kept as related OA for continuous verification language.

5. Research question, hypotheses, and solution

RQ: How can a unified tenant-aware framework integrate JWT tenant claims, middleware filtering, PostgreSQL RLS, and Redis multi-dimensional quotas to achieve zero unauthorized cross-tenant access, Jain fairness above 0.9, authentication under 50 ms, and quota overhead under 5 ms?

Hypotheses (directional): H1 — defence-in-depth blocks all synthetic cross-tenant reads in the probe suite; H2 — Jain fairness stays above 0.9 under the pilot multi-tenant workload; H3 — auth p95 stays under 50 ms and quota p95 under 5 ms in the floor harness; H4 — hard quota caps reject overflow while soft caps only warn.

Solution: one Python harness implements all four layers; synthetic tenants generate documents; middleware and RLS deny cross-tenant reads; quotas consume API/storage/compute atomically; a 10,000-probe suite attempts JWT tenant mismatch. Floor LOCAL_SIM pilot (20 tenants × 100 requests + 10k probes): 0 leaks; Jain ≈ 0.993; auth and quota p95 far under budgets in-memory. Docker Compose ships real Postgres RLS SQL for the next gate.

6. Dataset / data

Synthetic tenant IDs (tenant-000 ...) and document rows only. No personal data and no production customer tenancy. Generator: `sim.runner.run_pilot`. Probe vectors are limited to researcher-controlled local processes (token tenant mismatch, RLS miss). Quota dimensions: `api`, `storage`, `compute`. Results under `results/local_sim/` labelled mode `local_sim`.

7. Implementation + technologies / AWS mapping

- Python: PyJWT, middleware, in-memory RLS store, atomic quota engine, fairness metrics, pen-test harness.
- `sql/rls_policies.sql` — `ENABLE/FORCE ROW LEVEL SECURITY` with `app.current_tenant` GUC.
- `docker-compose.yml` — Postgres 16 + Redis 7 (optional; floor tests do not require docker).
- AWS mapping (future): Cognito/API Gateway JWT, Lambda/ECS middleware, RDS PostgreSQL RLS, ElastiCache Redis.
- Floor: make setup && make test && make pilot — eleven pytest cases passed on 2026-09-27.

8. End-to-end flow

- Issue JWT with `tenant_id` and role; verify signature and expiry on each request.
- Middleware extracts claims; `CrossTenantDenied` if resource tenant $\neq$ JWT tenant (unless `platform_admin`).
- SET session tenant; RLS-style store returns only matching rows; missing context raises.
- Quota engine `check_and_consume` on `api/storage/compute`; hard reject above cap; soft flag above soft.
- Run 10k cross-tenant probes; write `pilot_results.json`; never quote as live AWS.

9. Local pilot numbers you may quote (LOCAL_SIM only)

From `results/local_sim/pilot_results.json` on 2026-09-27: `cross_tenant_leaked` = 0 on 10,000 dedicated probes; `zero_unauthorized_access` = true; Jain fairness index $\approx$ 0.993; auth p95 $\approx$ 0.01 ms; quota p95 ■ 5 ms. These timings prove the control flow is cheap in-process; they do not replace a networked RDS/ElastiCache campaign. If asked for AWS evidence, point to `STATUS LIVE_AWS=not_run`.

Quota abuse sample in the same JSON shows hard caps reject after the configured API limit. Soft limits flag without hard-fail so operators can alert before customers are cut off. Jain remains high when tenants run similar request loops under their caps; unequal enterprise caps are a policy choice documented in `configs/experiment.yaml`.

10. Exact speaking script (dataset + implementation + technologies)

My dataset is synthetic tenants and documents — no personal data. I propagate tenant identity from a JWT through application middleware, a row-level security style database session, and Redis-like multi-dimensional quotas. I ran ten thousand cross-tenant probes locally and recorded zero unauthorized access, with Jain fairness about 0.99 and auth and quota checks well under the fifty and five millisecond budgets in-memory. My baseline is Cheng et al. 2025 at VLDB on fair transaction scheduling; their gap is the scheduler, while I evaluate the end-to-end SaaS enforcement chain my CA required. Docker Compose is ready for Postgres and Redis; live AWS is not claimed yet.

11. Five likely Q&A

Q: Is this live on AWS RDS/ElastiCache?

A: No. Floor is LOCAL_SIM; docker-compose is optional; LIVE_AWS=not_run in STATUS.

Q: Is Cheng et al. still the right baseline?

A: Yes — VLDB 2025, CA1-named, OA PDF present; gap is end-to-end JWT+RLS+quotas vs txn fairness.

Q: Do you use real customer tenant data?

A: No — synthetic tenants only; GDPR-minimising evaluation design.

Q: What stops a forgotten WHERE clause?

A: Defence in depth: middleware deny plus RLS session GUC so the database still filters.

Q: Where is the artefact?

A: pavan/tenant-aware-enforcement/ — make test && make pilot.

12. Validity, STATUS lock, and honesty rules

Internal validity: same synthetic generator and probe set; enforcement layers are the treatment. External validity: in-memory latency understates network hops; FORCE RLS on Postgres must be re-validated under connection pooling. Construct validity: unauthorized access counts successful reads of another tenant's row; Jain uses per-tenant API usage shares.

STATUS one-liner: FLOOR COMPLETE — runnable local artefact, eleven tests passed, Cheng et al. 2025 baseline documented, evaluation LOCAL_SIM only. Never invent AWS numbers, never commit secrets, never force-push. Prefer local folders and docs; open PRs only with clean one-commit branches when gh auth allows.

13. Governance takeaway for viva

Tenant isolation is not only a reliability feature; it is a data-protection control. The CA report argues that designing for real customer data makes GDPR Article 5(1)(f), Article 25, and Article 32 applicable even when the evaluation uses synthetic tenants. Publishing the enforcement framework with synthetic datasets supports open science and reproducibility, while probe details stay high-level so dual-use risk stays managed. Energy and log-retention concerns also matter: every request verifies JWT, applies RLS, and touches

Redis counters, so enforcement must stay cheap — which the floor latency budgets illustrate directionally.

If the examiner asks what remains after floor: (1) run docker Postgres RLS integration tests against `sql/rls_policies.sql`; (2) map Cognito/API Gateway + RDS + ElastiCache on the student AWS account; (3) destroy stacks after campaigns; (4) keep STATUS honest if live numbers differ from LOCAL_SIM.